

R&D Trivy and Gitleaks CI/CD Security Integration Report

Department: DevOps

Prepared by: Thanusha Bai V

Date: 123-sept-2026

Task: DEV-493-R&D Trivy & Gitleaks on CI/CD Pipeline

1. Executive Summary

This report documents the research, integration, and validation of two open-source security tools – **Trivy** (container vulnerability scanning) and **Gitleaks** (secret detection) – into an existing Jenkins CI/CD pipeline.

The objective was to define a secure, scalable scanning strategy that catches vulnerabilities and leaked secrets **before** artifacts reach production deployments (Nomad).

A proof-of-concept was built using **Jenkins, Harbor, Trivy, and Gitleaks**, all running in a containerized environment.

Both tools were verified to:

- Detect their respective target threats (**CVEs and hardcoded secrets**).
- Block the CI/CD pipeline when threats are found.
- Integrate cleanly with existing DevOps workflows (**GitHub → Jenkins → Harbor**).

Three-Layer Security Gate Architecture

Layer	Tool	Purpose
CI – Pre-build	Gitleaks	Block on secrets in source code
CI – Post-build	Trivy	Block on HIGH/CRITICAL CVEs in image
Registry	Harbor + Trivy	Independent verification + pull prevention

Both security gates were proven to work – the pipeline **failed as designed** when a fake secret and vulnerable image were introduced.

2. Objectives

The task required:

1. **Research Trivy capabilities** – image scanning, filesystem/dependency scanning, Harbor and CI integration.
2. **Research Gitleaks capabilities** – hardcoded secret detection, pre-commit and CI scanning modes.
3. **Design a CI/CD integration flow** – determine when scans run and what happens on failure.
4. **Perform a proof-of-concept** – validate detection against real images and repositories.
5. **Evaluate output formats** – JSON, SARIF, and compatibility with dashboards.
6. **Document findings and recommendations.**

3. Tool Research

3.1 Trivy

Overview

Trivy is an open-source, all-in-one security scanner maintained by Aqua Security. It detects vulnerabilities in OS packages, language-specific dependencies, misconfigurations, and secrets.

Capabilities

Feature	Supported
Container image scanning (OS + app packages)	✓
Filesystem & dependency scanning	✓
Infrastructure-as-Code (IaC) scanning	✓
Secret detection (extended)	✓
SBOM generation (SPDX, CycloneDX)	✓
Multi-source vulnerability DB (NVD, GHSA, Red Hat, Alpine, Debian, etc.)	✓
Output formats: table, JSON, SARIF, JUnit, template	✓
Harbor integration (default scanner)	✓
Kubernetes Operator	✓

Scan Modes Relevant to This Task

Mode	Use Case
trivy image <image>	Scan container images before deployment
trivy fs <path>	Scan source directory for dependency CVEs
trivy repo <url>	Scan a remote Git repository

Key Flags Used

```
trivy image \  
  --severity HIGH,CRITICAL \  
  --exit-code 1 \  
  --no-progress \  
  --format json \  
<image>:<tag>
```

Flag	Purpose
--severity HIGH,CRITICAL	Only fail on serious vulnerabilities
--exit-code 1	Fail the pipeline when findings match
--no-progress	Clean output for CI logs
--format json	Machine-readable output for downstream tools
--ignore-unfixed	Skip CVEs without available patches (optional)

3.2 Gitleaks

Overview

Gitleaks is a fast, open-source SAST tool for detecting hardcoded secrets in Git repositories and filesystems. It uses a library of regex patterns combined with entropy analysis.

Capabilities

Feature	Supported
Source code secret detection	✓
Git history scanning	✓
Filesystem/directory scanning	✓
Pre-commit hook mode	✓
CI/CD pipeline mode	✓
Custom rules (.gitleaks.toml)	✓
Multiple output formats (JSON, SARIF, CSV)	✓
~100+ built-in secret detectors	✓

Scan Modes Relevant to This Task

Mode	Command	Use Case
Directory scan	gitleaks dir /path	Scan current working tree (used in this PoC)
Git history	gitleaks git /path	Scan commit history (best for PRs)
Pre-commit	gitleaks protect --staged	Runs locally before commit

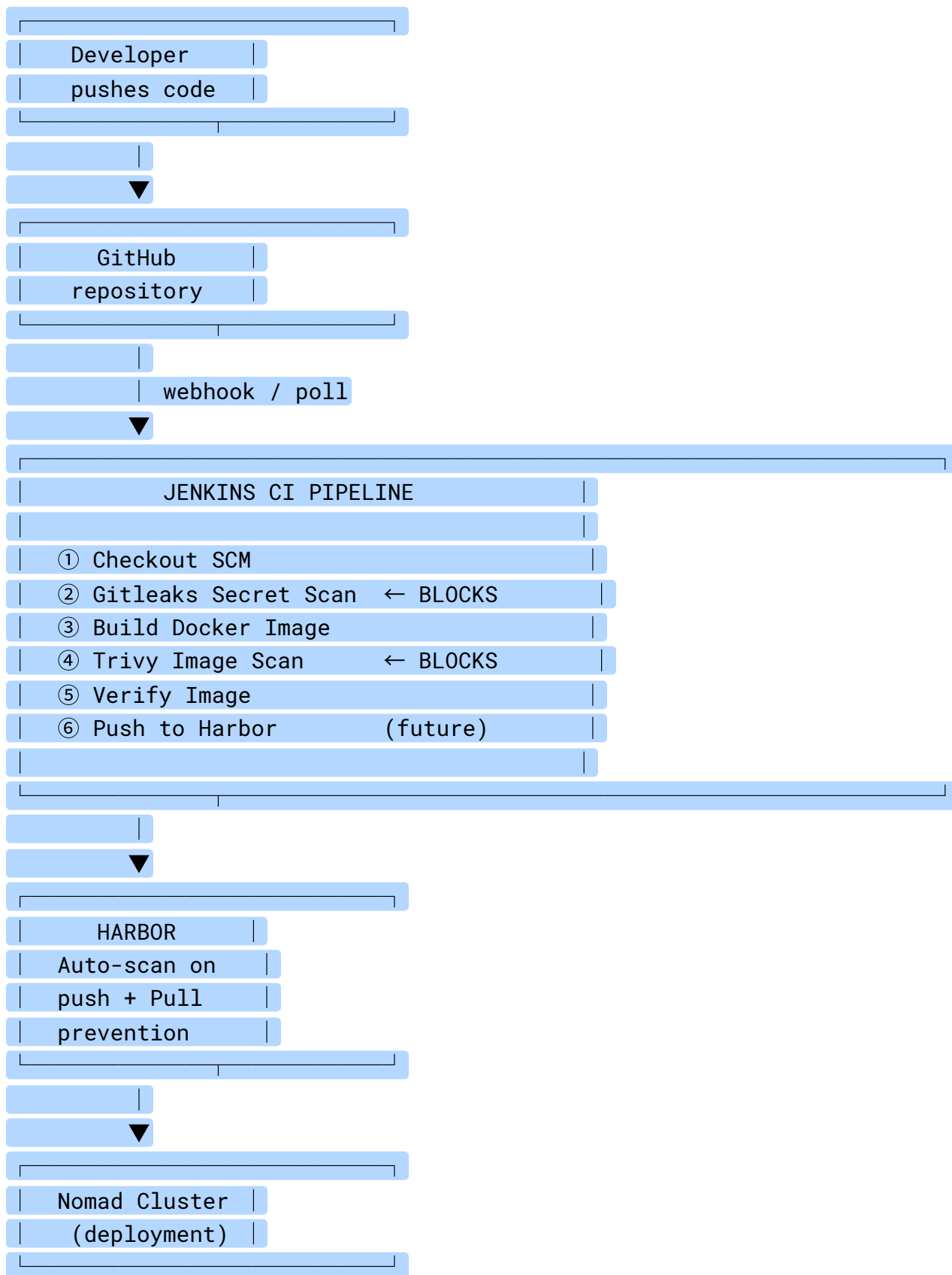
Key Flags Used

```
gitleaks dir /src \  
  --verbose \  
  --redact \  
  --exit-code=1
```

Flag	Purpose
dir /src	Scan a directory (not Git history)
--verbose	Print findings with context
--redact	Mask secret values in output
--exit-code=1	Fail the pipeline when secrets are found

4. Integration Design

4.1 Architecture Flow



4.2 Scan Timing – Rationale

Timing	Tool	Reason
Pre-build	Gitleaks	Cheapest to fail early – before any artifact is created
Post-build	Trivy	Image must exist before it can be scanned
Pre-deploy	Harbor	Independent verification + catches newly disclosed CVEs
Pre-commit (local)	Gitleaks	Fastest feedback for developers (recommended addition)

4.3 Failure Policies

Gate	Severity	Action
Gitleaks	Any secret	✗ Block build
Trivy (CI)	HIGH or CRITICAL	✗ Block build
Trivy (CI)	MEDIUM or LOW	⚠ Warn only (log)
Harbor pull prevention	HIGH or CRITICAL	✗ Prevent deployment
Harbor pull prevention	MEDIUM or LOW	✓ Allow

Design principle: Block only on **high-confidence, high-impact findings** to avoid pipeline fatigue.

5. Proof of Concept

5.1 Environment

Component	Details
Host	Ubuntu VM (VirtualBox)
RAM	3.3 GB usable + 2 GB swap
Disk	30 GB (12 GB free)
Docker	29.1.3
Jenkins	2.568.3 (custom image custom-jenkins:lts-jdk21)
Harbor	v2.15.2 (with Trivy adapter)
Trivy	v0.72.0
Gitleaks	v8 (ghcr.io/gitleaks/gitleaks:latest)

5.2 Gitleaks – PoC

Test Procedure

1. Created test-secret.env containing a high-entropy fake GitHub token:

```
GITHUB_TOKEN=ghp_<36-random-characters>
```

2. Committed the file to GitHub.
2. Triggered the Jenkins build.

Result

✔ BUILD BLOCKED

Detection Output

```
Finding:      GITHUB_TOKEN=REDACTED
Secret:      REDACTED
RuleID:      github-pat
Entropy:     4.671928
File:        /src/test-secret.env
Line:        2
Fingerprint: /src/test-secret.env:github-pat:2
|
leaks found: 1
ERROR: script returned exit code 1
Finished: FAILURE
```

Downstream Impact

- Build Docker Image stage – **skipped**
- Verify Image stage – **skipped**
- No artifact was produced

Key Insight

A high-entropy random token, rather than a documentation placeholder, is required to trigger detection. Gitleaks' entropy analysis filters out placeholder patterns.

Evidence: See **Appendix A – Screenshot 4**.

5.3 Trivy (CI) – PoC

Test Procedure

1. Built sample-app image from node:20-alpine base.
2. Trivy scanned the image with --severity HIGH,CRITICAL.
3. Later, upgraded the base to node:22-alpine and re-tested.

Result – node:20-alpine

✔ BUILD BLOCKED

Detection Summary

Layer	Vulnerabilities
Alpine OS (3.23.4)	4 HIGH
Node.js packages	19 HIGH + 1 CRITICAL
Total	24 (1 CRITICAL, 23 HIGH)

Critical Finding

CVE-2026-59873 CRITICAL

Package: tar

Installed: 6.2.1

Fixed: 7.5.19

Title: Denial of Service via crafted gzip bomb

Exit Code: 1 → Build FAILED

Downstream Impact

- Verify Image stage – **skipped**
- Pipeline completed with status: **FAILURE**

Result – node:22-alpine

CVEs were reduced to 11 (1 CRITICAL, 10 HIGH), demonstrating how base image upgrades reduce attack surface.

Evidence: See Appendix A – Screenshots 5 and 6.

5.4 Harbor (Registry) – PoC

Setup

- Harbor v2.15.2 installed with --with-trivy
- Project sample-app created
- Policies enabled:
 -  Automatically scan images on push
 -  Prevent vulnerable images from running (severity: **High**)
- Docker configured with insecure-registry for Harbor

Test Procedure

1. Pushed sample-app:v1 to Harbor:

192.168.0.9/sample-app/sample-app:v1

2. Harbor automatically triggered the Trivy scan.
2. Viewed the vulnerability report.

Result

 79 VULNERABILITIES DETECTED

Severity	Count
Critical	1
High	78
Total	79 (all fixable)

Critical Finding Identified

CVE-2026-59873 – tar – gzip bomb DoS

Registry Policies Confirmed

- Every image push → auto-scan
- Images with HIGH+ CVEs → prevented from running
- Vulnerability report viewable per-image in Harbor UI

Evidence: See Appendix A – Screenshots 7, 8, and 9.

6. Output Formats & Tooling Compatibility

6.1 Trivy Output Formats

Format	Use Case	Notes
table (default)	Human-readable CLI	Best for CI logs
json	Machine-readable	Best for custom dashboards and automation
sarif	GitHub Security tab, IDEs	Standard for security findings
junit	Jenkins test reports	Useful for CI dashboards
template	Custom formatting	Requires Go template
cyclonedx, spdx	SBOM generation	For compliance workflows

Recommendation: Use `--format json` in CI for downstream processing, and `--format sarif` for GitHub Security integration.

6.2 Gitleaks Output Formats

Format	Use Case
json (default when reporting)	Parsing, dashboards
sarif	GitHub Security tab
csv	Spreadsheet review

Recommendation: Export sarif and upload to GitHub's Code Scanning alerts.

6.3 Dashboard Compatibility

Tool	Integration
GitHub Security tab	Gitleaks + Trivy via SARIF
Jenkins	Console output + JUnit plugin
Harbor	Built-in vulnerability UI
DefectDojo	Import Trivy JSON
Grafana / ELK	Via JSON export

7. Findings

7.1 Gitleaks

Strengths

- ✔ Fast scan times (~100 ms on small repositories)
- ✔ High-entropy detection avoids false positives on placeholder secrets
- ✔ Multi-language pattern library (AWS, GitHub, Stripe, Slack, private keys, etc.)
- ✔ Portable Docker image – zero installation
- ✔ Works equally well as a pre-commit hook or CI stage

Limitations

- ⚠ Requires custom rules (.gitleaks.toml) for organization-specific patterns
- ⚠ Only scans text – binaries ignored
- ⚠ Entropy-based detection may skip low-entropy secrets such as short passwords
- ⚠ Does not scan for secrets inside container images (only source)

7.2 Trivy

Strengths

-  Multi-source vulnerability database – broad coverage
-  Fast with cached DB – a few seconds per image
-  Native Harbor integration – zero additional setup
-  Supports OS + language packages in one scan
-  Multiple output formats including SARIF
-  Extensible to IaC, SBOM, and secrets scanning

Limitations

-  No reachability analysis – all CVEs reported, even if not exploitable
-  Requires network access to update the database unless offline mode is used
-  First run is slow due to DB download (~50 MB)
-  Cannot scan from within a container without Docker socket mount

7.3 Harbor

Strengths

-  Independent verification layer – catches images not built in CI
-  Periodic re-scanning detects newly disclosed CVEs on existing images
-  Project-level policy control – per-team severity thresholds
-  Built-in RBAC, robot accounts for CI, LDAP/OIDC
-  Registry-level pull prevention via orchestrator integration

Limitations

-  Resource-heavy – ~2-3 GB RAM for full stack
-  Pull prevention requires an orchestrator admission controller and is not enforced by docker pull
-  Requires TLS in production; HTTP is only acceptable for PoC

8. Tool Comparison

8.1 Container Scanners

Feature	Trivy	Grype	Clair	Snyk
License	Apache 2.0	Apache 2.0	Apache 2.0	Commercial
OS package scanning	✓	✓	✓	✓
Language packages	✓	✓	⚠ Partial	✓
IaC scanning	✓	✗	✗	✓
Secret scanning	✓	✗	✗	✓
SBOM generation	✓	✓	✗	✓
Harbor integration	✓ Native	⚠ Pluggable	✓ Deprecated	⚠ Pluggable
Kubernetes Operator	✓	✗	✗	✓
Performance	Fast	Fast	Slow	Fast (cloud)
Air-gapped	✓	✓	✓	✗
Ease of use	★★★★★	★★★★★	★★★★	★★★★★

Conclusion

Trivy covers image scanning, filesystem scanning, IaC, SBOM, and secrets, and integrates natively with Harbor. Grype is an alternative but lacks Harbor-native integration and secret scanning.

8.2 Secret Scanners

Feature	Gitleaks	TruffleHog	detect-secrets	GitGuardian
License	MIT	AGPL-3.0	Apache 2.0	Commercial
Regex + entropy	✓	✓	✓	✓
Git history scan	✓	✓	⚠ Limited	✓
Pre-commit hook	✓	⚠	✓	✗
CI mode	✓	✓	✓	✓
Custom rules	✓	✓	✓	✓
Speed	Fast	Medium	Fast	Fast (cloud)
Docker image	✓	✓	⚠	✗
SARIF output	✓	✓	⚠	✓

Conclusion

Gitleaks provides an open-source option with an MIT license, fast scanning, Docker packaging, and support for both pre-commit and CI workflows. TruffleHog provides stronger detection depth but uses an AGPL license. GitGuardian is SaaS with a limited free tier.

9. Recommendations

9.1 Adoption Strategy

Priority	Recommendation	Timeline
P0	Adopt Gitleaks as pre-commit hook + CI gate	Immediate
P0	Adopt Trivy in CI with HIGH/CRITICAL blocking	Immediate
P1	Adopt Harbor with auto-scan + pull prevention	Short-term
P2	Add SARIF output → GitHub Security tab	Medium-term
P2	Implement documented .trivyignore policy	Medium-term
P3	Add image signing (Cosign) + SBOM generation	Long-term
P3	Nomad admission controller for Harbor integration	Long-term

9.2 Security Gate Configuration

Gate 1 – Gitleaks (Pre-build)

```

|
|
| GATE 1 – GITLEAKS (pre-build) |
| Condition: any secret detected |
| Action:  ✖ BLOCK build        |
|
|
|
```

↓

Gate 2 – Trivy (Post-build)

GATE 2 – TRIVY (post-build)	
Condition:	HIGH or CRITICAL CVE
Action:	✗ BLOCK build

↓

Gate 3 – Harbor (Registry)

GATE 3 – HARBOR (registry)	
Condition:	HIGH or CRITICAL CVE on pushed image
Action:	✗ PREVENT pull / deploy

9.3 Operational Guidelines

- **Threshold tuning:** Block on HIGH/CRITICAL. Warn on MEDIUM. Ignore LOW in CI while still reporting in Harbor.
- **Exception handling:** Use .trivyignore and .gitleaks.toml allowlists for known, documented risks, reviewed quarterly.
- **DB refresh:** Schedule daily Trivy database updates to catch newly disclosed CVEs.
- **False positive triage:** Assign a security champion to review and tune rules monthly.
- **Base image hygiene:** Prefer alpine/distroless bases and rebuild images periodically to pick up patches.

10. Limitations of This PoC

Area	PoC State	Production Requirement
Harbor TLS	HTTP only	HTTPS with valid certificates
Harbor DB	Local PostgreSQL	External managed DB
Jenkins auth	Local user	SSO / LDAP / OIDC
Jenkins agents	Single agent	Distributed build agents
Credentials	Jenkins credential store	HashiCorp Vault / external secrets
Scanning resource	3.3 GB RAM VM	Dedicated scanning nodes
Nomad integration	Not implemented	Admission controller + Harbor webhook
Automation triggers	Manual "Build Now"	GitHub webhook / polling

11. Conclusion

The R&D objectives have been **fully met**:

- ☒ **Trivy** was successfully integrated into the CI/CD pipeline and verified to block builds on HIGH/CRITICAL CVEs.
- ☒ **Gitleaks** was successfully integrated and verified to block builds on hardcoded secrets.
- ☒ **Harbor** provides a third security layer with auto-scan on push and pull prevention policies.
- ☒ A **three-layer security gate architecture** was defined, tested, and proven.
- ☒ **Tool comparison** was documented – Trivy and Gitleaks were evaluated against Grype, Clair, Snyk, TruffleHog, and detect-secrets.
- ☒ **Output formats** such as JSON and SARIF were evaluated for dashboard integration.

Both tools were identified as **production-ready for adoption** within the scope of this PoC.

The documented strategy is to roll out:

1. **Gitleaks** for secret detection.
2. **Trivy** in CI for container vulnerability scanning.
3. **Harbor** as an additional registry-level security gate.

Appendix A – Evidence Screenshots

Screenshot 1 – Jenkins Build History

 Jenkins

All

Build History

+ New Item

Build History

Project Relationship

Check File Fingerprint

Build Queue

No builds in the queue.

Build Executor Status

(0 of 2 executors busy)

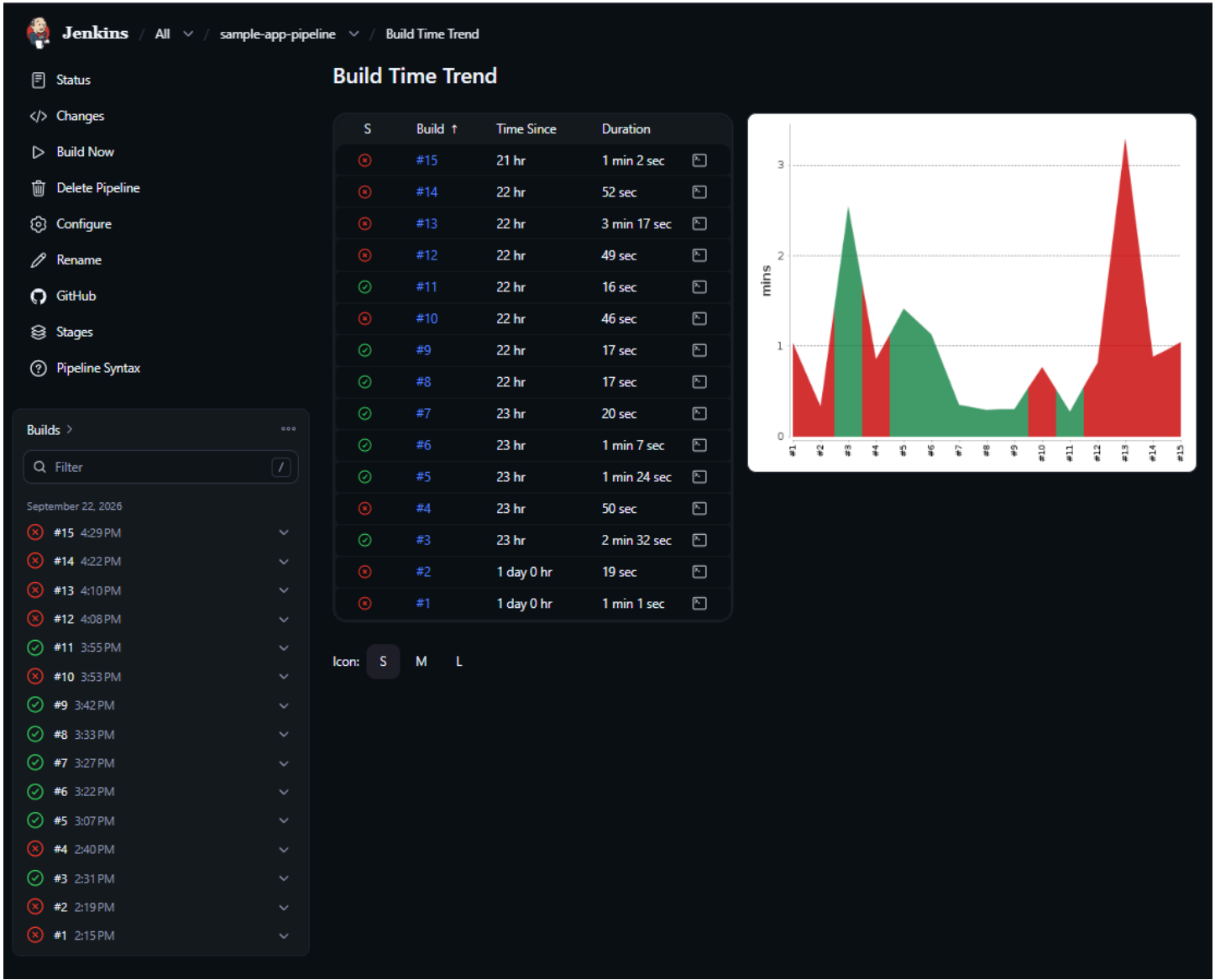
Build History of Jenkins

S	Build	Time Since	f	Status
	sample-app-pipeline #15	21 hr		broken since build #12
	sample-app-pipeline #14	22 hr		broken since build #12
	sample-app-pipeline #13	22 hr		broken since build #12
	sample-app-pipeline #12	22 hr		broken since this build
	sample-app-pipeline #11	22 hr		back to normal
	sample-app-pipeline #10	22 hr		broken since this build
	sample-app-pipeline #9	22 hr		stable
	sample-app-pipeline #8	22 hr		stable
	sample-app-pipeline #7	22 hr		stable
	sample-app-pipeline #6	23 hr		stable
	sample-app-pipeline #5	23 hr		back to normal
	sample-app-pipeline #4	23 hr		broken since this build
	sample-app-pipeline #3	23 hr		back to normal
	sample-app-pipeline #2	1 day 0 hr		broken for a long time
	sample-app-pipeline #1	1 day 0 hr		broken since this build

Icon: S M L

Full build history showing 15 builds with pass/fail states.

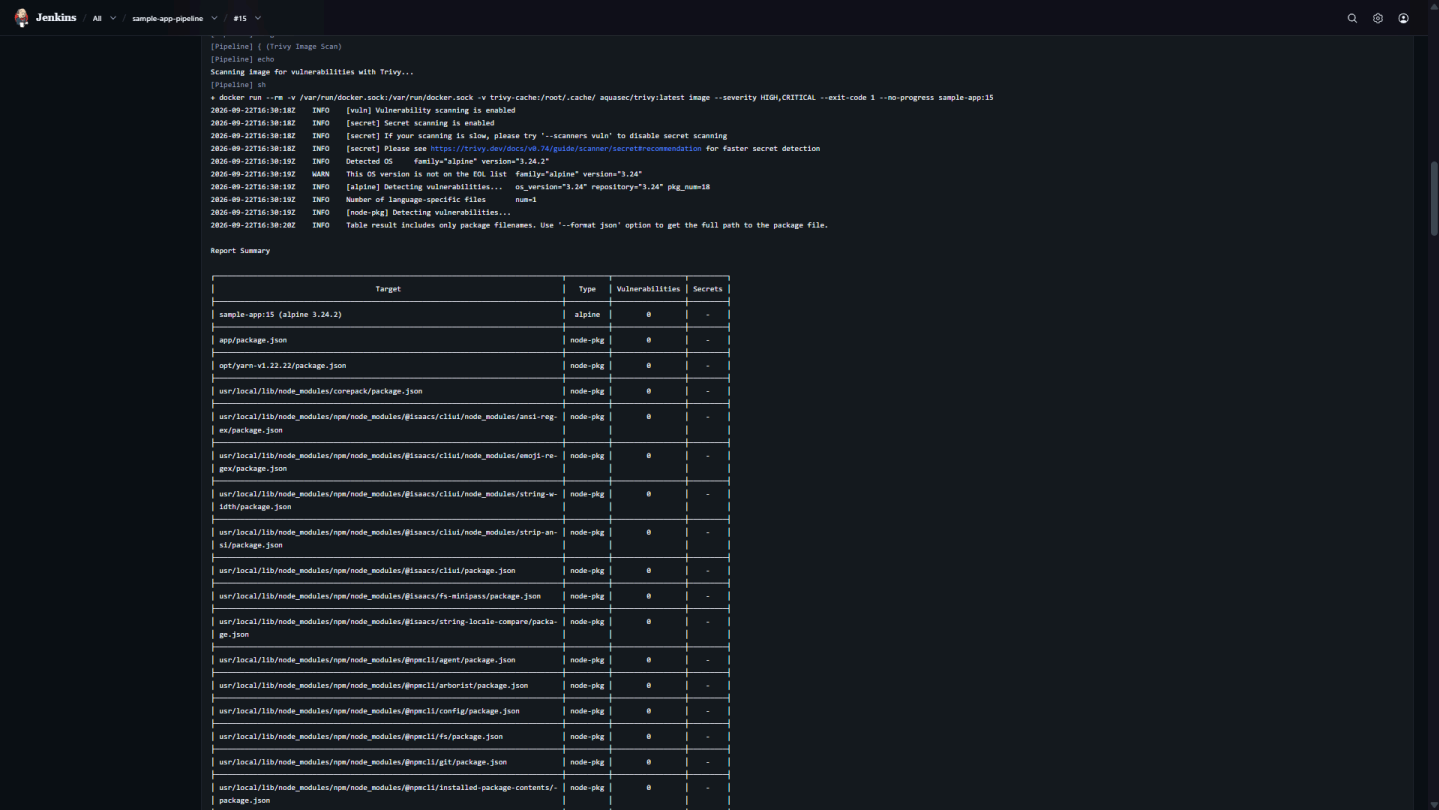
Screenshot 2 — Build Time Trend



Build duration chart across all runs.

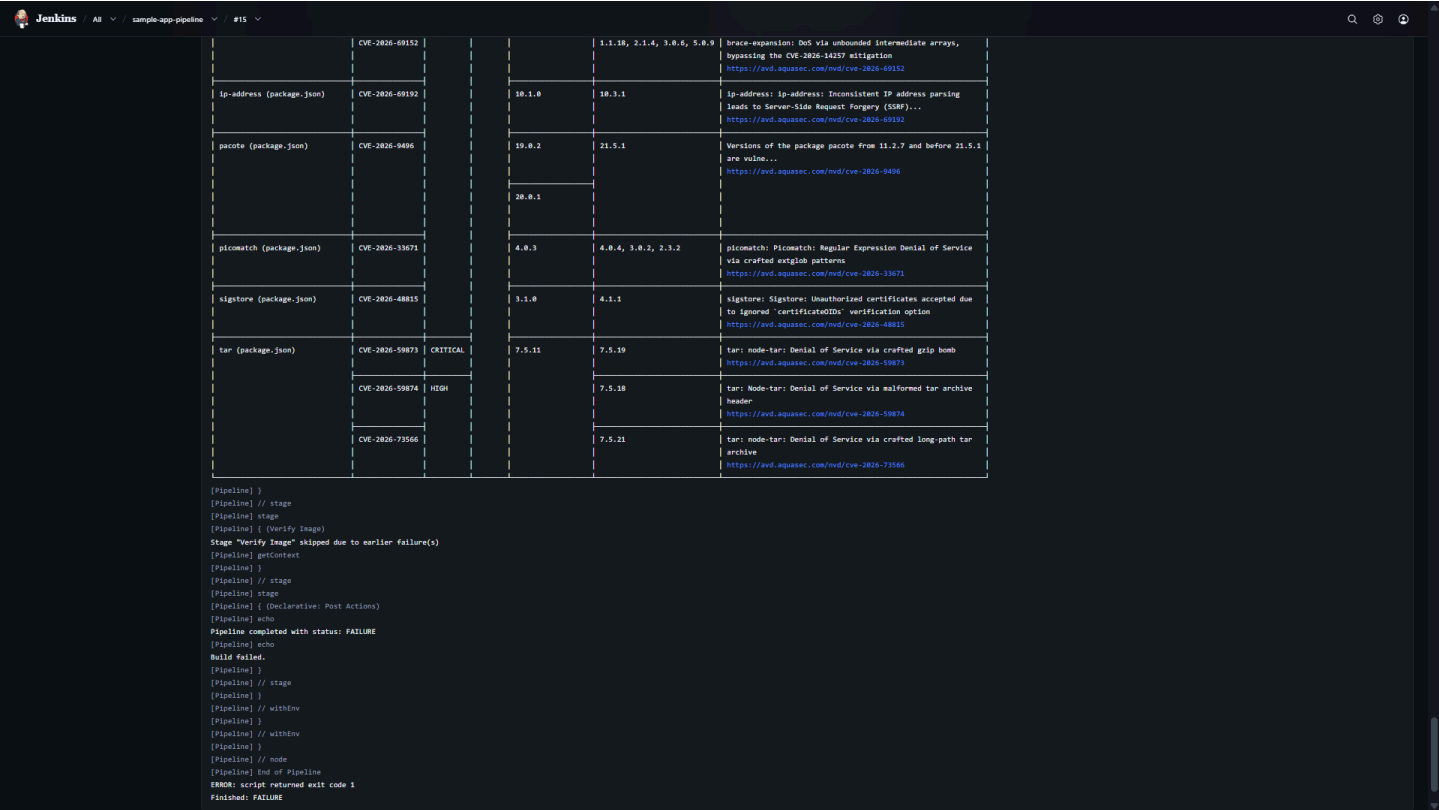
Screenshot 3 — Pipeline Stages

Screenshot 5 – Trivy Command



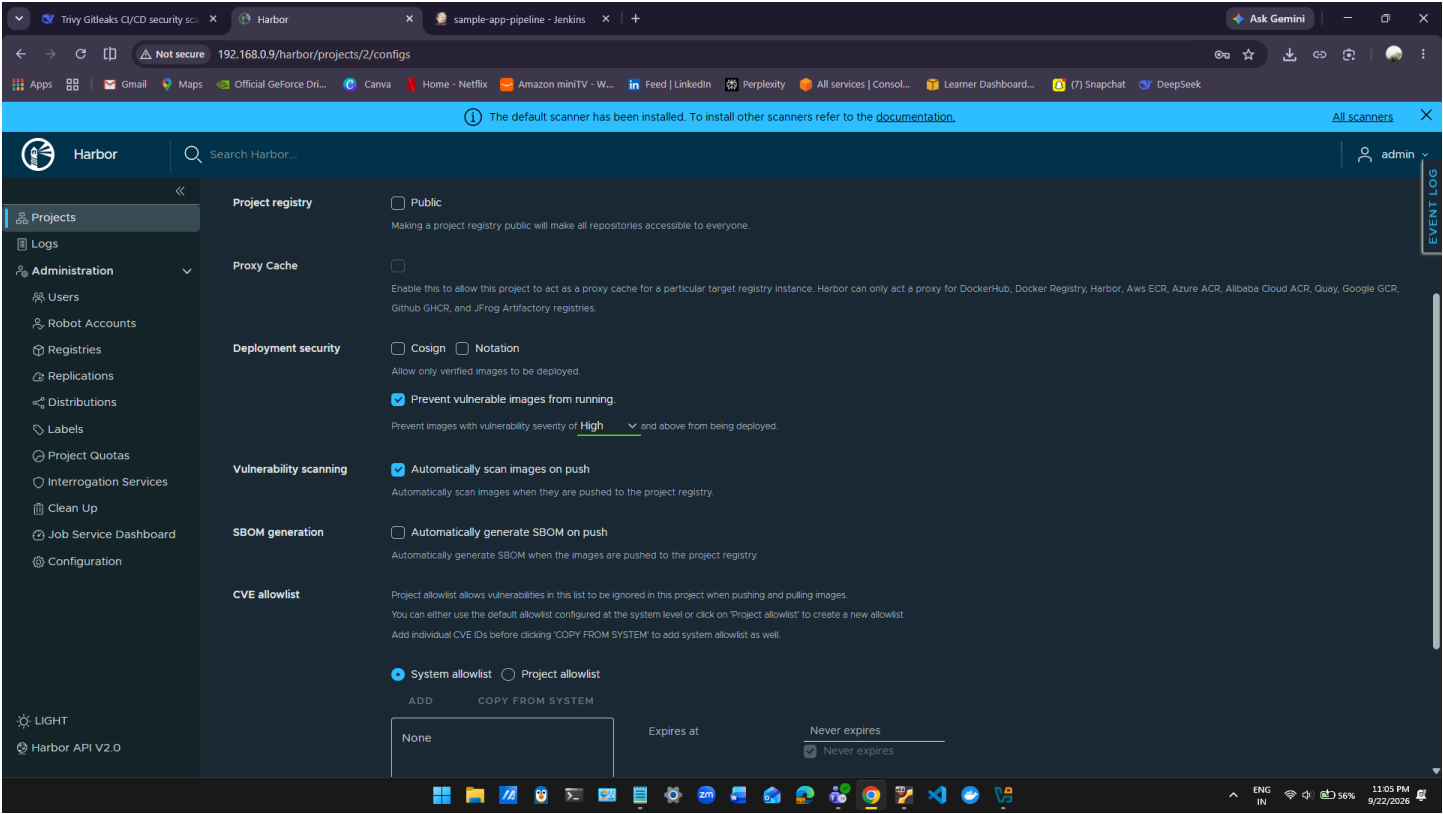
Trivy scan command executed within the Jenkins pipeline.

Screenshot 6 – Trivy Blocking Build



Critical CVE detected; pipeline blocked before deployment.

Screenshot 7 – Harbor Project Policies



Harbor project configured with auto-scan and pull prevention at High severity.

Screenshot 8 – Harbor Trivy Vulnerabilities

Vulnerability	Severity	CVSS	Status	Package	Current version	Fixed in version	Listed in CVE Allowlist
CVE-2026-59873	Critical	ghsa: rnvdt: redhat: 7.5	fixed	tar	6.2.1	7.5.19	No
CVE-2026-14456	High	redhat: 7.5	fixed	libcrypto3	3.5.6-r0	3.5.8-r0	No
CVE-2026-45447	High	julix: redhat: 8.8	fixed	libcrypto3	3.5.6-r0	3.5.7-r0	No
CVE-2026-14456	High	redhat: 7.5	fixed	libssl3	3.5.6-r0	3.5.8-r0	No
CVE-2026-45447	High	julix: redhat: 8.8	fixed	libssl3	3.5.6-r0	3.5.7-r0	No
CVE-2024-29538	High	ghsa: redhat: 7.5	fixed	cross-spawn	7.0.3	7.0.5, 6.0.6	No
CVE-2026-13149	High	ghsa: redhat: 5.3	fixed	brace-expansion	2.0.1	5.0.7, 11.16, 21.2	No
CVE-2026-14257	High	ghsa: redhat: 7.5	fixed	brace-expansion	2.0.1	5.0.8, 3.0.3, 21.3, 11.17	No
CVE-2026-69152	High	ghsa: redhat: 7.5	fixed	brace-expansion	2.0.1	11.18, 21.4, 3.0.6, 5.0.9	No
CVE-2025-84756	High	ghsa: redhat: 7.5	fixed	glob	10.4.2	11.10, 10.5.0	No
CVE-2026-69192	High	ghsa: redhat: 8.6	fixed	ip-address	9.0.5	10.3.1	No
CVE-2026-26996	High	ghsa: rnvdt: redhat: 7.5	fixed	minimatch	9.0.5	10.2.1, 9.0.6, 8.0.5, 7.4.7, 6.2.1, 5.1.7, 4.2.4, 3.1.3	No
CVE-2026-27903	High	ghsa: redhat: 7.5	fixed	minimatch	9.0.5	10.2.3, 9.0.7, 8.0.6, 7.4.8, 6.2.2, 5.1.8, 4.2.5, 3.1.3	No
CVE-2026-27904	High	ghsa: redhat: 7.5	fixed	minimatch	9.0.5	10.2.3, 9.0.7, 8.0.6, 7.4.8, 6.2.2, 5.1.8, 4.2.5, 3.1.4	No
CVE-2026-9496	High	ghsa: 7.5	fixed	pacote	18.0.6	21.5.1	No

Harbor's built-in Trivy scan showing 79 CVEs including 1 Critical.

Screenshot 9 – Harbor Artifacts Page

Artifacts	Tags	Signed	Size	Vulnerabilities	SBOM	Labels	Push Time	Pull Time
sha256:7caf215	v1		46.13MiB	79 Total - 79 Fixable	No SBOM		9/22/26, 11:25 PM	9/23/26, 12:07 AM

Artifact list showing "79 Total - 79 Fixable" vulnerability badge.

Appendix B – References

- **Trivy Documentation:** <https://trivy.dev/>
- **Gitleaks GitHub Repository:** <https://github.com/gitleaks/gitleaks>
- **Harbor Documentation:** <https://goharbor.io/docs/>
- **Jenkins Pipeline Syntax:** <https://www.jenkins.io/doc/book/pipeline/syntax/>
- **GitHub Actions Marketplace – Trivy:** <https://github.com/marketplace/actions/aqua-security-trivy>
- **OWASP DevSecOps Guideline:** <https://owasp.org/www-project-devsecops-guideline/>

End of Report